CSE 331: Computer Security Fundamentals

Spring 2026

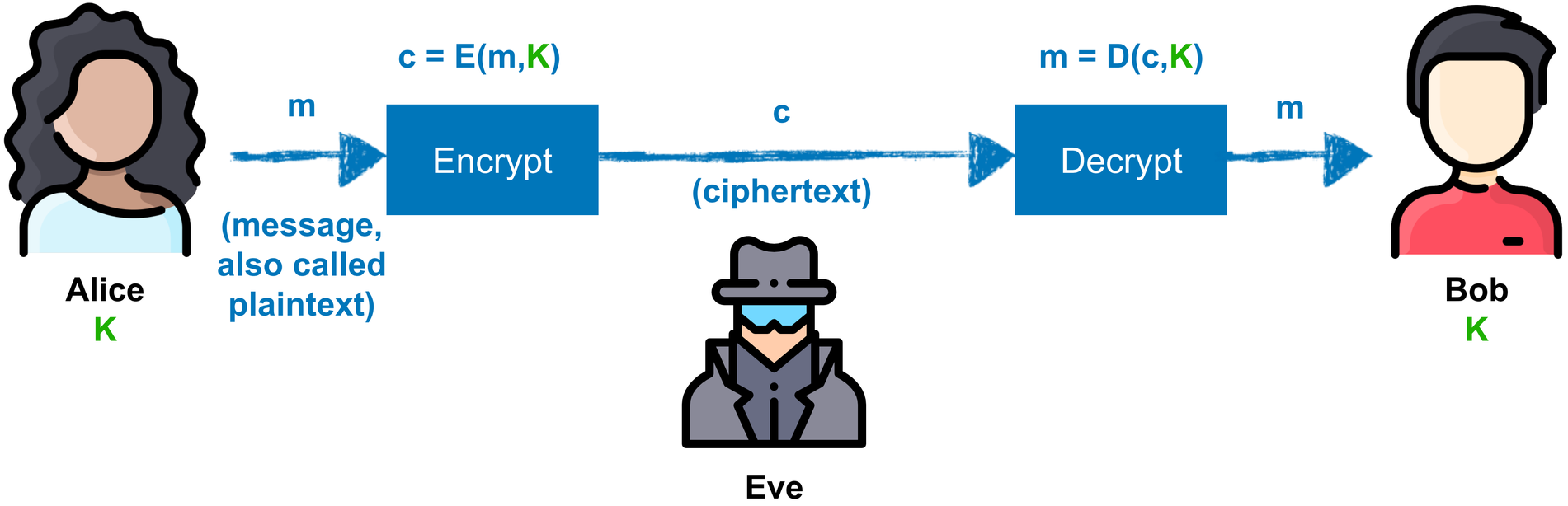
Asymmetric Encryption:
DH Key Exchange, RSA Basics

Amir Rahmati
Xinrui Yu

Stony Brook Institute
at Anhui University
安徽大学纽约石溪学院

Review: Symmetric Setting

Both communicating parties have access to a shared random string K, called the key.



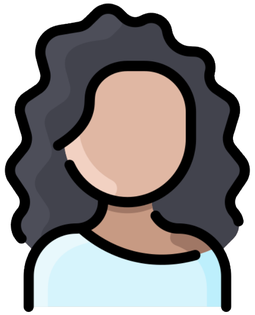
Problem: How do you share a key? You can't encrypt it!

Basic Problem

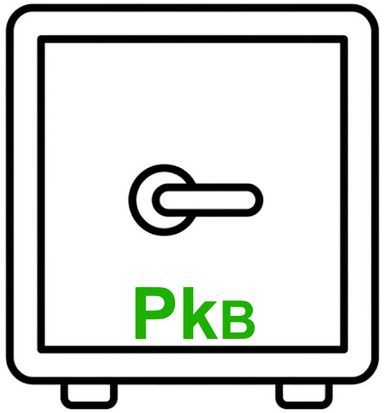
- Given:
    - Everybody knows Bob's public key
    - Only Bob knows their private key
- Goals:
    - Alice wants to send a secret message to Bob
    - Alice wants to authenticate herself

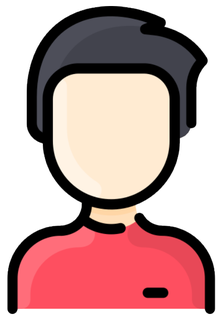
Asymmetric Setting

Each party creates a public key Pk and a secret key Sk.



Alice





Bob

SkA

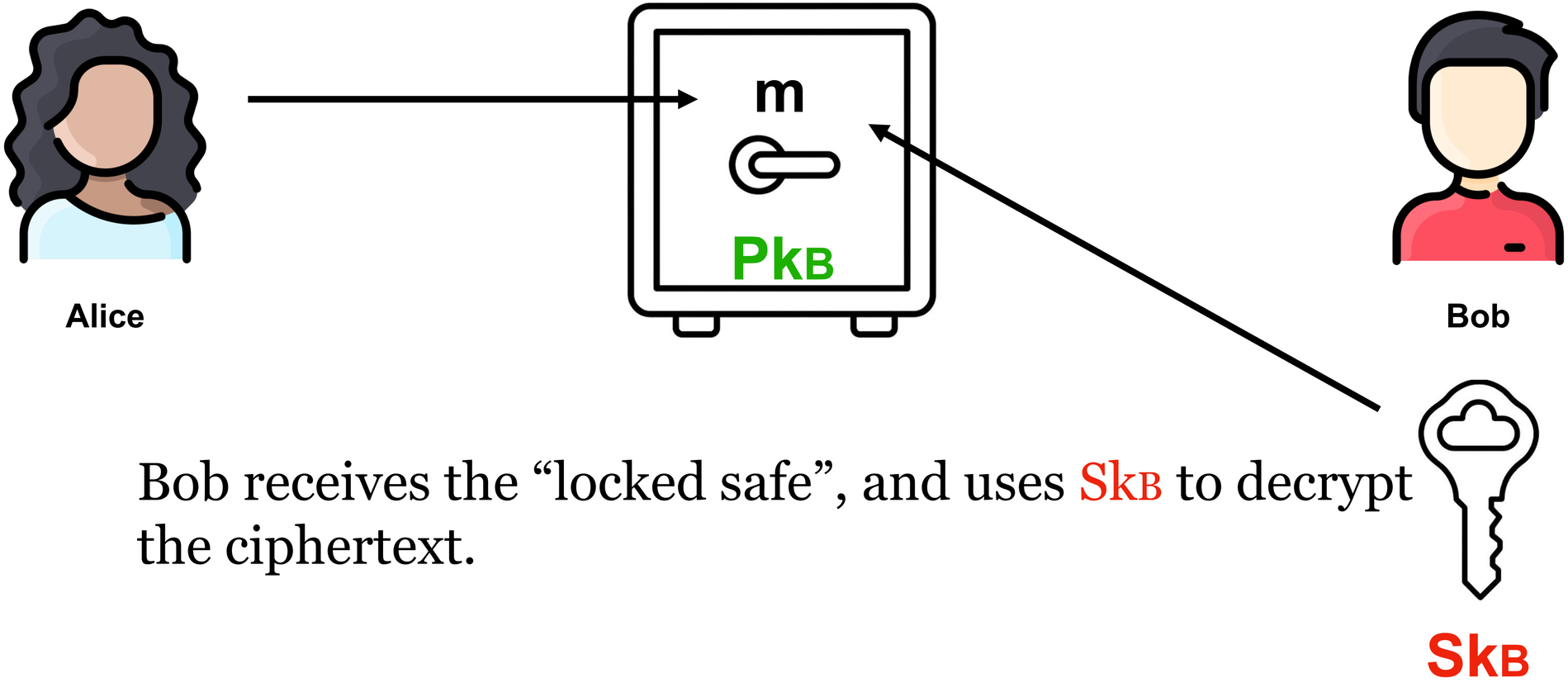
Public keys are shared (everyone has access)

Only the owner knows its secret key.

SkB

Encryption Analogy

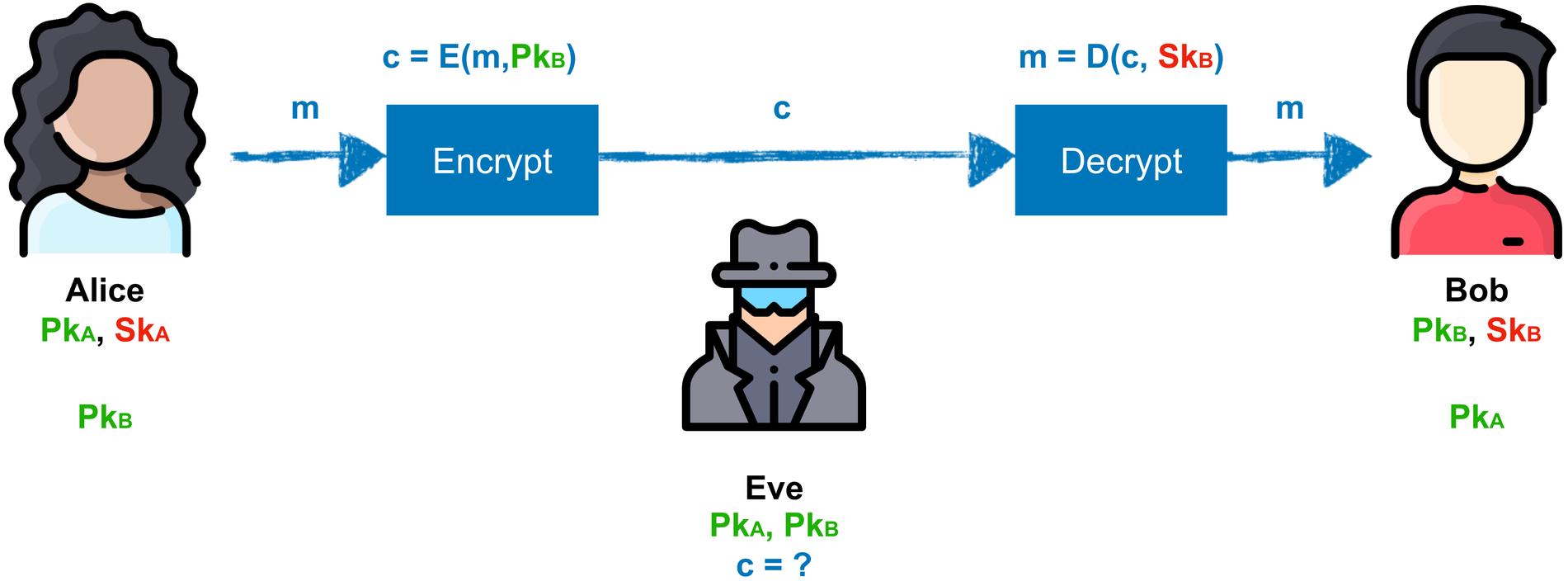
Alice wants to send a message to Bob. Alice uses Bob's public key Pk_B to encrypt the message.



Bob receives the "locked safe", and uses Sk_B to decrypt the ciphertext.

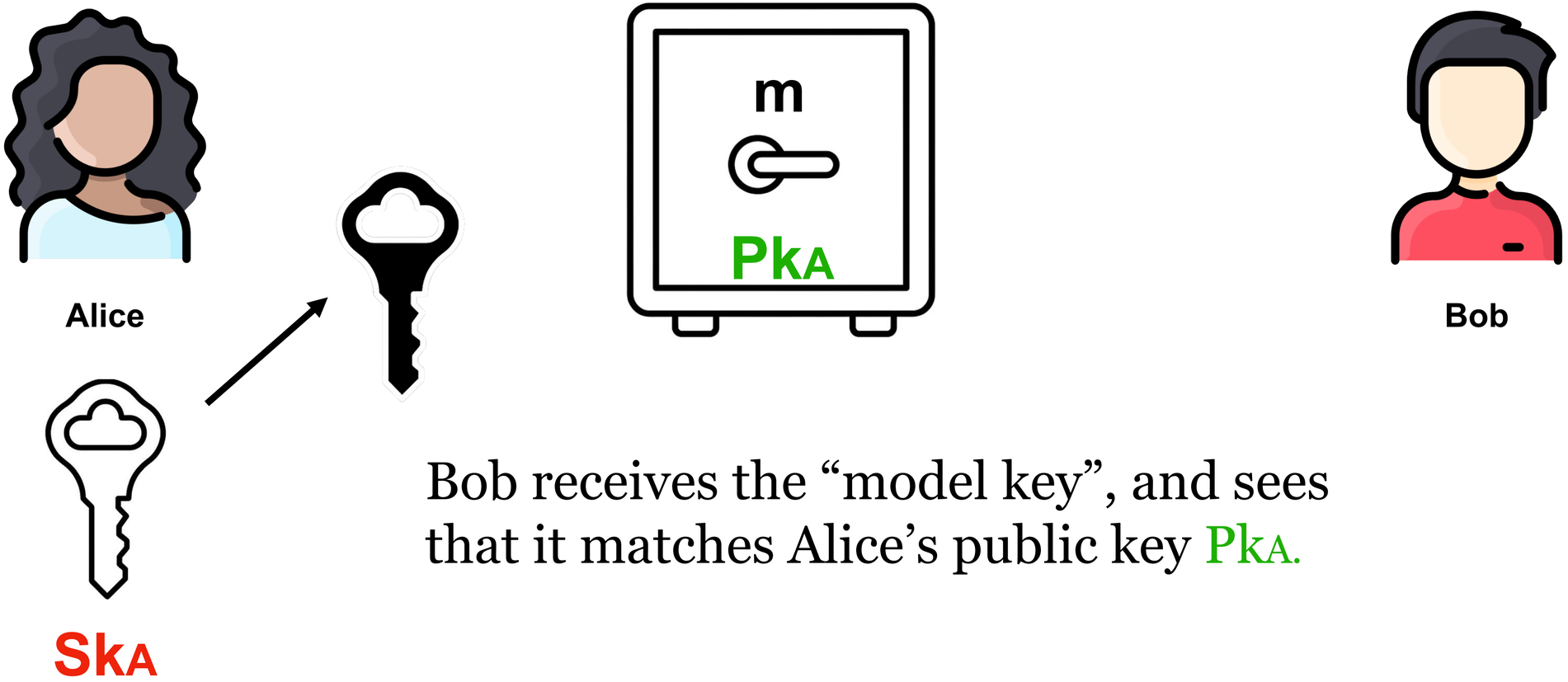
Asymmetric Setting

Each party creates a public key Pk and a secret key Sk.



Authenticate Analogy

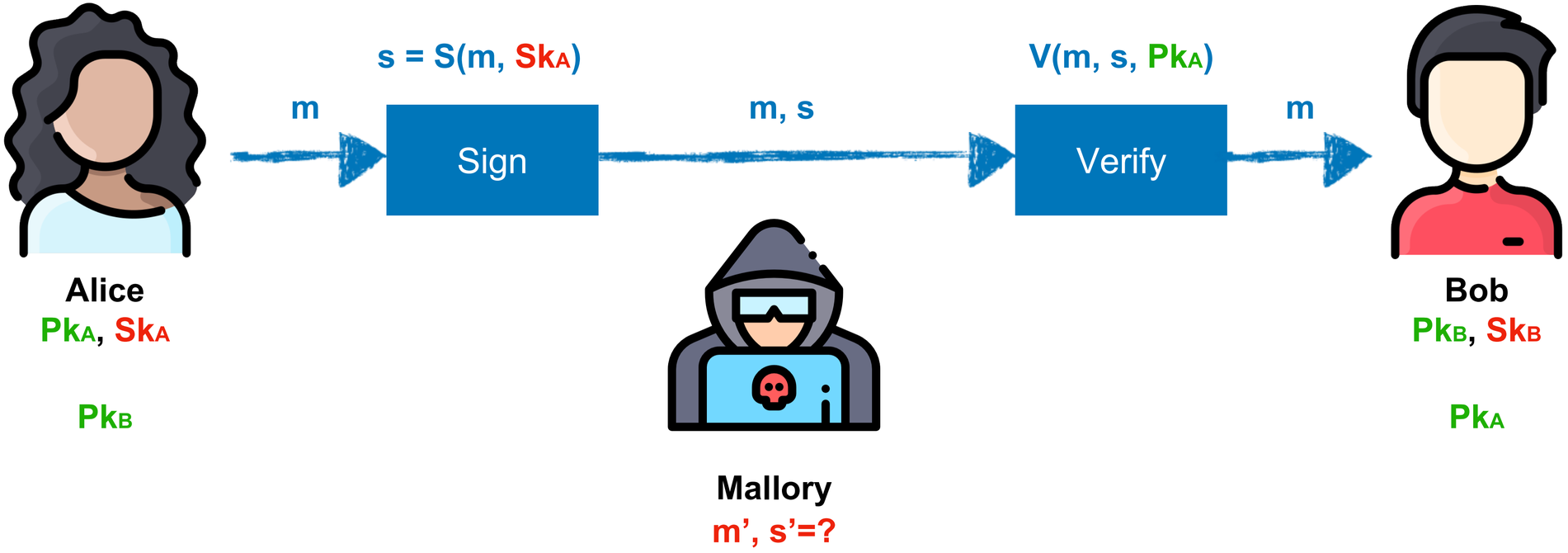
Alice wants her identity to be verified by Bob. She creates a “model key” S (signature) with her Sk_A.



Bob receives the “model key”, and sees that it matches Alice’s public key Pk_A.

Asymmetric Setting

Each party creates a public key Pk and a secret key Sk.



Applications of Public Key Crypto

- Encryption for confidentiality
  - Anyone can encrypt a message
  - Only someone who knows private key can decrypt
  - Key management is simpler (or at least different)
    - Secret is stored only at one site: good for open environments
- Digital signatures for authentication
  - Can "sign" a message with your private key
- Session key establishment
  - Exchange messages to create a secret session key
  - Then switch to symmetric cryptography (why?)

Mostly because that asymmetric encryption is slow and inefficient.

Session Key Establishment

- I want to securely communicate with some person on the other side of the world.
- We never meet in person.
- We have not shared any secret key a priori. (as opposed to a posteriori)
- Using only information passed in a public setting, at the end of the protocol, we will have learnt a secret that ONLY the two of us know.

-> We need a **Key Exchange Protocol**.

Key Exchange Protocol Example: Three Primary School Students

Say three primary school students learned **multiplication** but **not division**.

Two of them want to have a **common secret number** between them without being known by the third.

They **publicly** speak out an integer such as 23 and agree upon this number, then each come up with a **secret** number, say A has 5, B has 3.

Now, A speak out the multiplication result of the publicly available number and his/her secret number 115, and B does the same and says 69.

Now, each of them can multiply the number said by the other and their own secret numbers, say A has 69 * 5 = 345, and B also has 115 * 3 = 345.

As a result, A and B now share a **common secret number** 345, while C, **overhearing the entire conversation**, cannot figure out the secret number because he or she has no knowledge on division.

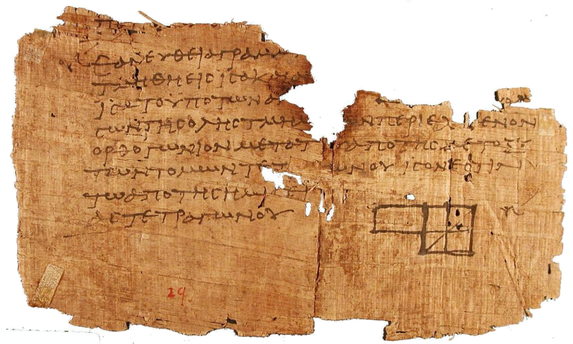
Assumption

We made a very important assumption: The three primary school students learned **multiplication** but **not division**.

While this is extreme, we did know that we want the following properties for an operation in **Key Exchange Protocol**:

1. The operation itself is relatively easy.
2. The inverse operation should be extremely hard.
3. (a * p) * b = a * (p * b) -> Associativity

Some Math



Euclid, Elements (300BC)

Fundamental theorem of arithmetic (unique factorization theorem)

"every integer greater than 1 either is a prime number itself or can be represented as the product of prime numbers and that, moreover, this representation is unique."

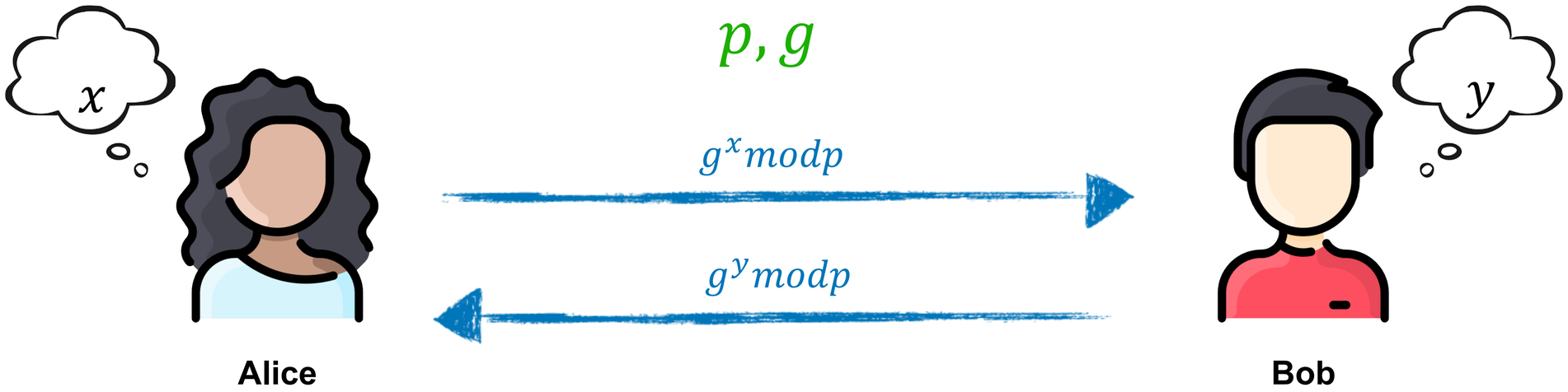
Prime Factorization: $1200 = 2^4 \times 3^1 \times 5^2$

Prime factorization is unique

Diffie-Hellman Protocol (1976)

- Alice and Bob never met and share no secrets
- Alice and Bob <u>publicly</u> agree on p and g
  - p is a large safe prime, g is a generator of Z_p*
    - Z_p*={1, 2 … p-1}; for-all a in Z_p* there-is i such that a=g^i mod p

Detailed mathematical discussion included for your reference will not be in the exam



Compute $k = (g^y)^x modp = g^{xy} modp$

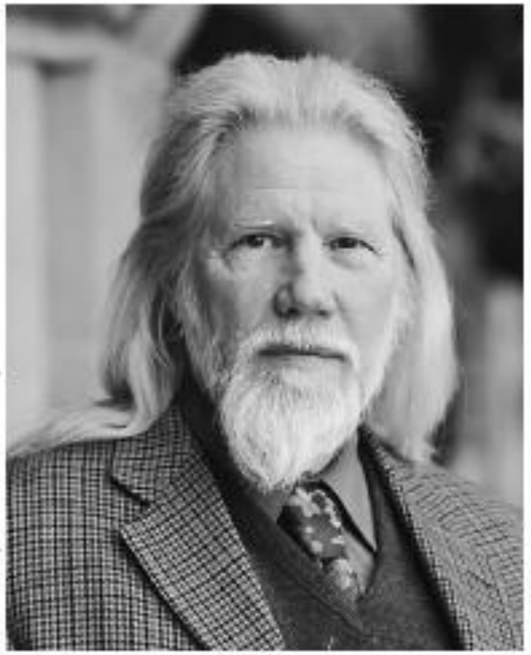
Compute $k = (g^x)^y modp = g^{xy} modp$

Associativity

Properties of Diffie-Hellman

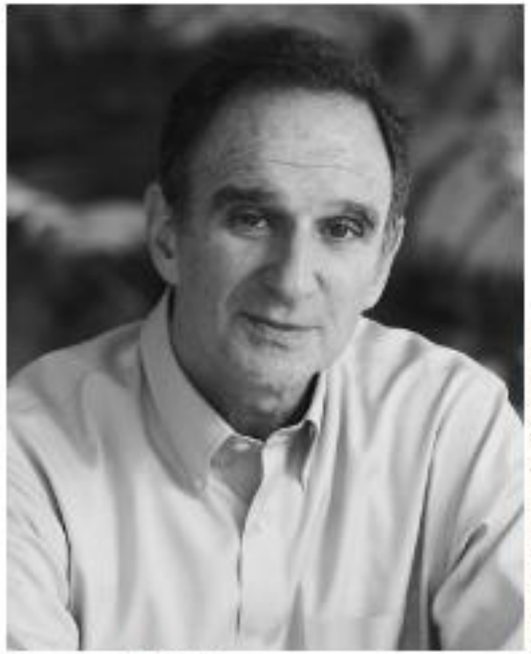
- Diffie-Hellman protocol is a secure key establishment protocol against <u>passive</u> attackers
    - Often hash $g^{xy} \bmod p$, and use the hash as the key
    - Can use the new key for symmetric cryptography
- Diffie-Hellman protocol (by itself) does not provide authentication
    - Party in the middle attack (often called “man in the middle attack”)

Diffie and Hellman Receive 2015 Turing Award



Rod Searcey/Stanford University.

Whitfield Diffie



Linda A. Cicero/Stanford News Service.

Martin E. Hellman

Applications of Public Key Crypto

- Encryption for confidentiality
  - Anyone can encrypt a message
  - Only someone who knows private key can decrypt
  - Key management is simpler (or at least different)
    - Secret is stored only at one site: good for open environments
- Digital signatures for authentication
  - Can “sign” a message with your private key
- Session key establishment
  - Exchange messages to create a secret session key
  - Then switch to symmetric cryptography (why?)

Requirements for Public Key Encryption

- Key generation: computationally easy to generate a pair (public key PK, private key SK)
- Encryption: given plaintext M and public key PK, easy to compute ciphertext $C=E_{PK}(M)$
- Decryption: given ciphertext $C=E_{PK}(M)$ and private key SK, easy to compute plaintext M
  - Infeasible to learn anything about M from C without SK
  - Trapdoor function: Decrypt(SK,Encrypt(PK,M))=M

RSA Cryptosystem

[Rivest, Shamir, Adleman 1977]

Detailed mathematical discussion included for your reference will not be in the exam

- Key generation:
  - Generate large primes p, q
    - Say, 1024 bits each (need primality testing, too)
  - Compute $n=pq$ and $\varphi(n)=(p-1)(q-1)$
  - Choose small e, relatively prime to $\varphi(n)$
    - Typically, $e=3$ or $e=2^{16}+1=65537$
  - Compute unique d such that $ed \equiv 1 \bmod \varphi(n)$
    - Modular inverse: $d \equiv e^{-1} \bmod \varphi(n)$
  - Public key = (e,n); private key = (d,n)
- Encryption of m: $c = m^e \bmod n$
- Decryption of c: $c^d \bmod n = (m^e)^d \bmod n = m$

$$d = \frac{k\phi(n)+1}{e}$$

Why is RSA Secure?

- Factoring problem: given positive integer n, find primes $p_1, \ldots, p_k$ such that $n=p_1^{e_1}p_2^{e_2}\ldots p_k^{e_k}$
  - If factoring is easy, then RSA problem is easy
  - Currently factoring algorithms are already much better than brute forcing, but still insufficient

Variable Key Length

- Largest (publicly known) factored RSA modulus is 829 bits long
- February 2020: took roughly 2700 core-years
- It is believed that 1024-bit keys may already (or in the near future) be breakable by a sufficiently powerful attacker
- 2048-bit keys should be the absolute minimum

https://en.wikipedia.org/wiki/RSA_Factoring_Challenge

RSA-150	150	496		April 16, 2004	Kazumaro Aoki *et al.*
RSA-155	155	512	US$9,383[8]	August 22, 1999	Herman te Riele *et al.*
RSA-160	160	530		April 1, 2003	Jens Franke *et al.*, University of Bonn
RSA-170 [b]	170	563		December 29, 2009	D. Bonenberger and M. Krone [c]
RSA-576	174	576	US$10,000	December 3, 2003	Jens Franke *et al.*, University of Bonn
RSA-180 [b]	180	596		May 8, 2010	S. A. Danilov and I. A. Popovyan, Moscow State University[11]
RSA-190 [b]	190	629		November 8, 2010	A. Timofeev and I. A. Popovyan
RSA-640	193	640	US$20,000	November 2, 2005	Jens Franke *et al.*, University of Bonn
RSA-200 [b] ?	200	663		May 9, 2005	Jens Franke *et al.*, University of Bonn
RSA-210 [b]	210	696		September 26, 2013[12]	Ryan Propper
RSA-704 [b]	212	704	US$30,000	July 2, 2012	Shi Bai, Emmanuel Thomé and Paul Zimmermann
RSA-220 [b]	220	729		May 13, 2016	S. Bai, P. Gaudry, A. Kruppa, E. Thomé and P. Zimmermann
RSA-230 [b]	230	762		August 15, 2018	Samuel S. Gross, Noblis, Inc.
RSA-232 [b]	232	768		February 17, 2020[13]	N. L. Zamarashkin, D. A. Zheltkov and S. A. Matveev.
RSA-768 [b]	232	768	US$50,000	December 12, 2009	Thorsten Kleinjung *et al.*[14]
RSA-240 [b]	240	795		Dec 2, 2019[15]	F. Boudot, P. Gaudry, A. Guillevic, N. Heninger, E. Thomé and P. Zimmermann
RSA-250 [b]	250	829		Feb 28, 2020[16]	F. Boudot, P. Gaudry, A. Guillevic, N. Heninger, E. Thomé and P. Zimmermann
RSA-260	260	862			
RSA-270	270	895			

Applications of Public Key Crypto

- Encryption for confidentiality
  - Anyone can encrypt a message
  - Only someone who knows private key can decrypt
  - Key management is simpler (or at least different)
    - Secret is stored only at one site: good for open environments
- Digital signatures for authentication
  - Can “sign” a message with your private key
- Session key establishment
  - Exchange messages to create a secret session key
  - Then switch to symmetric cryptography (why?)

RSA Signatures

- Public key is **(n,e)**, private key is **(n,d)**
- To sign message m: $s = m^d \bmod n$
  - Signing & decryption are same **underlying** operation in RSA
  - It's infeasible to compute **s** on **m** if you don't know **d**
- To verify signature s on message m:
  verify that $s^e \bmod n = (m^d)^e \bmod n = m$
  - Just like encryption (for RSA primitive)
  - Anyone who knows **n** and **e** (public key) can verify signatures produced with d (private key)
- In practice, also need padding & hashing
  - Standard padding/hashing schemes exist for RSA signatures

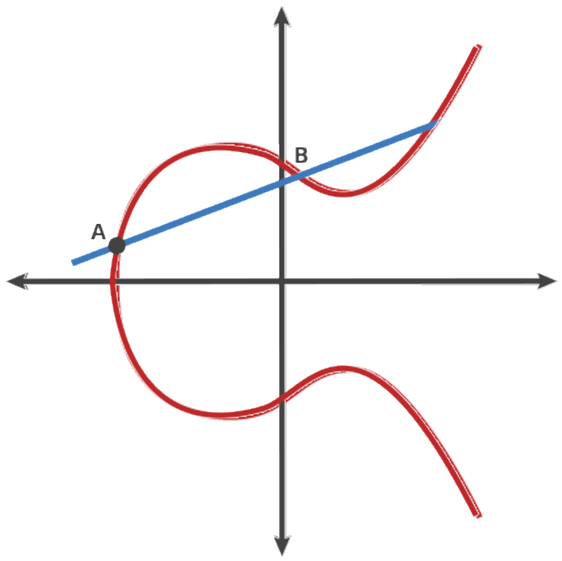
Elliptic Curve

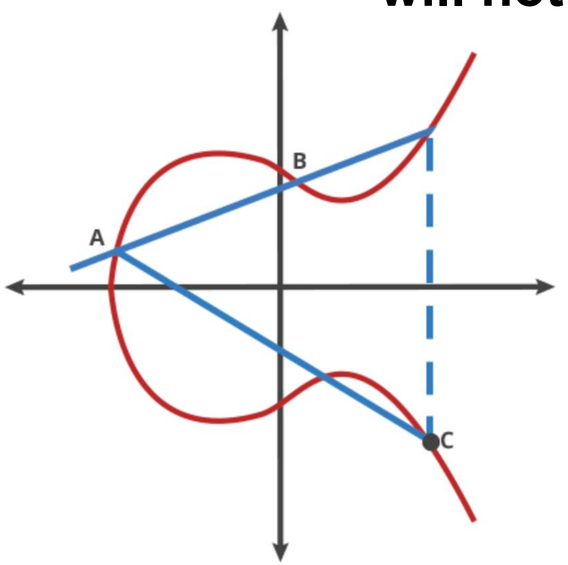
- Proposed in 1985, but not used until 15 years later
- Relies on the intractability of a different mathematical problem: elliptic curve discrete logarithm
- Endorsed by NIST
  - Key exchange: elliptic curve Diffie–Hellman (ECDH)
  - Digital signing: elliptic curve digital signature algorithm (ECDSA)

$y^2 = x^3 + ax + b$

Elliptic Curve

Detailed mathematical discussion included for your reference will not be in the exam





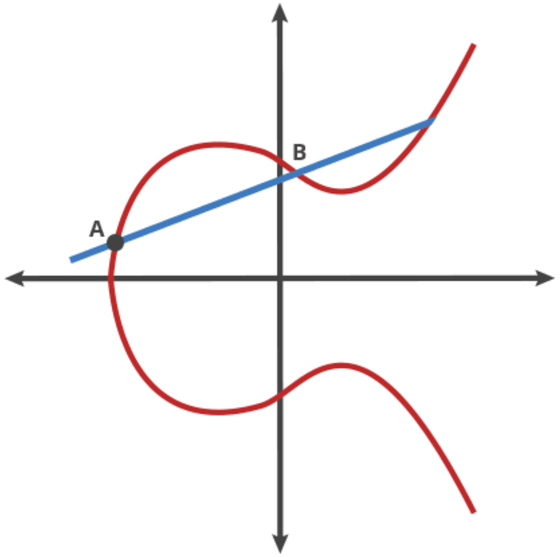
$$A.B = C$$

$$A.A = B$$
$$A.B = C$$
$$A.C = D$$
$$\ldots$$

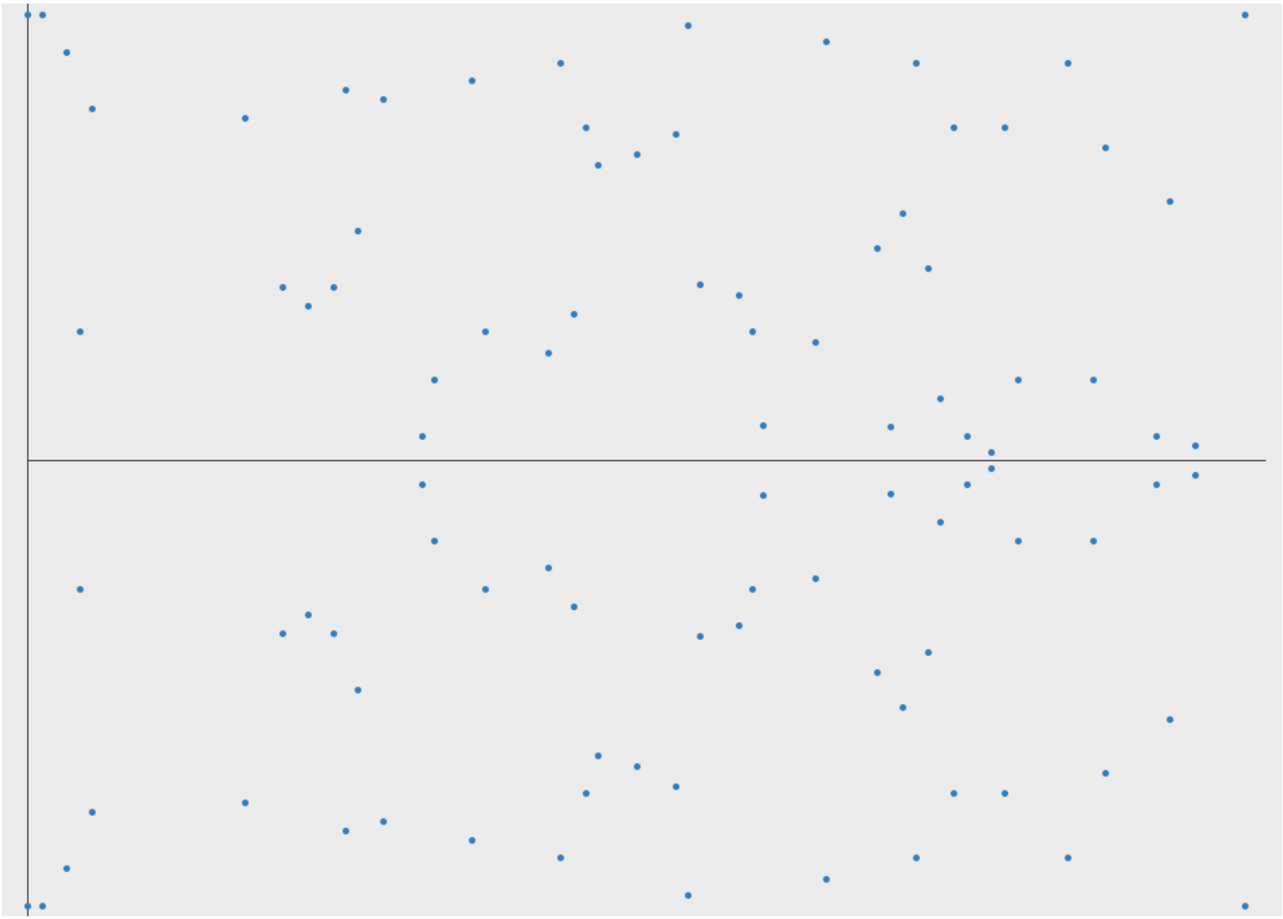
$$a(bA) = b(aA)$$

Elliptic Curve

$$y^2 = x^3 + ax + b$$



$$y^2 = x^3 - x + 1$$



max = 97 (modulo arithmetic)

Elliptic Curve

- Crypto system: Curve Equation, Prime number as max, public point on curve (G)
- Private key: a
- Public key: $A=aG$
- Encryption:
  - Pick random value b
  - Calculate bG , bA
  - send bG , $Encrypt_{bA}(m) = c$
- Decryption:
  - calculate $a(bG)$ = b(aG)=bA
  - $Decrypt_{bA}(c) = m$

Symmetric Key Size (bits)	RSA and Diffie-Hellman Key Size (bits)	Elliptic Curve Key Size (bits)
80	1024	160
112	2048	224
128	3072	256
192	7680	384
256	15360	521

Table 1: NIST Recommended Key Sizes

Mersenne Prime, 2^521 - 1

Commercial National Security Algorithm Suite and Quantum Computing FAQ

Q: What is the Commercial National Security Algorithm Suite?

A: The Commercial National Security Algorithm Suite is the suite of algorithms identified in CNSS Advisory Memorandum 02-15 for protecting NSS up to and including TOP SECRET classification. This suite of algorithms will be incorporated in a new version of the National Information Assurance Policy on the Use of Public Standards for the Secure Sharing of Information Among National Security Systems (CNSSP-15 dated October 2012). The Advisory Memorandum and Policy define the set of public cryptographic standards that may be used to

Algorithm	Usage
RSA 3072-bit or larger	Key Establishment, Digital Signature
Diffie-Hellman (DH) 3072-bit or larger	Key Establishment
ECDH with NIST P-384	Key Establishment
ECDSA with NIST P-384	Digital Signature
SHA-384	Integrity
AES-256	Confidentiality

Cryptography Summary

- Goal: Privacy
  - Symmetric keys:
    - One-time pad, Stream ciphers
    - Block ciphers (e.g., DES, AES) → modes: EBC, CBC, CTR
  - Public key crypto (e.g., Diffie-Hellman, RSA)
- Goal: Integrity
  - MACs, often using hash functions (e.g, MD5, SHA-256)
- Goal: Privacy and Integrity
  - Encrypt-then-MAC
- Goal: Authenticity (and Integrity)
  - Digital signatures (e.g., RSA)